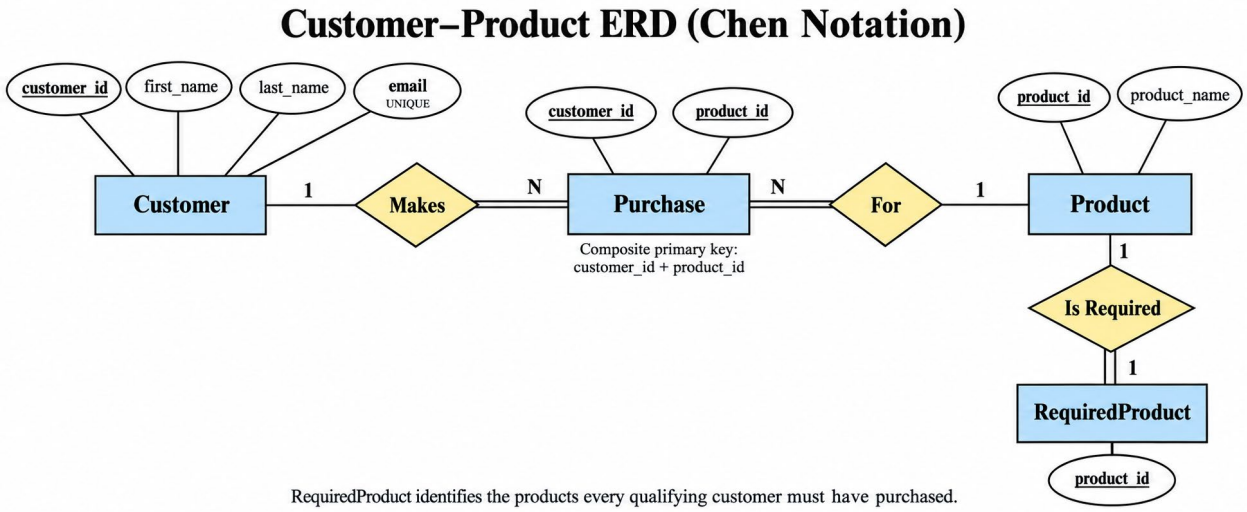
Division Style Query

A **division-style query** finds records related to **every item in a specified set**. It is based on *relational division*.

Relational division is a database operation used to answer questions involving **"all" or "every."**

For example:



Which customers have purchased **every required product**?

Suppose the required products are **A, B, and C**:

Customer	Products purchased	Included?
Alex	A, B, C	Yes
Jamie	A, B	No
Taylor	A, B, C, D	Yes

Alex and Taylor qualify because both purchased **all three required products**. Additional purchases are allowed.

Relational division compares two sets of information:

- **Purchase:** customer–product pairs showing who bought what.
- **RequiredProduct:** the products each qualifying customer must have purchased.

The result is the set of customer IDs associated with **every product in `RequiredProduct`**.

SQL has no standard `DIVIDE` keyword. You typically express relational division using **`NOT EXISTS`** or **`GROUP BY`** with **`HAVING`**.

For example:

Find customers who have purchased **every required product**.

Tables:

- Customer c (customer_id, first_name, last_name, email)
- Purchase p (customer_id, product_id)
- Product (product_id, product_name)
- RequiredProduct r (product_id)

One way to write the query is with two `NOT EXISTS` conditions:

```
SELECT c.customer_id, c.first_name, c.last_name
FROM Customer c
WHERE NOT EXISTS (
    SELECT 1
    FROM RequiredProduct r
    WHERE NOT EXISTS (
        SELECT 1
        FROM Purchase p
        WHERE p.customer_id = c.customer_id
          AND p.product_id = r.product_id
    )
);
```

Using **101, 102, and 103** as the required products:

Inner `NOT EXISTS`: "Is this required purchase missing?"

Customer	Required product	Purchase exists?	Inner `NOT EXISTS`
Alex	101	Yes	**FALSE**
Alex	102	Yes	**FALSE**
Alex	103	Yes	**FALSE**
Jamie	101	Yes	**FALSE**
Jamie	102	Yes	**FALSE**
Jamie	103	No	**TRUE**
Taylor	101	Yes	**FALSE**
Taylor	102	Yes	**FALSE**
Taylor	103	Yes	**FALSE**

Only rows where the inner `NOT EXISTS` is **TRUE** pass the middle query's `WHERE` condition. These represent **missing purchases**.

Outer `NOT EXISTS`: "Are there no missing required purchases?"

Customer	Inner results for 101, 102, 103	Rows returned by middle query	Outer `NOT EXISTS`	Included?
Alex	FALSE, FALSE, FALSE	0	**TRUE**	Yes
Jamie	FALSE, FALSE, TRUE	1	**FALSE**	No
Taylor	FALSE, FALSE, FALSE	0	**TRUE**	Yes

```
Alex   → No missing products → NOT EXISTS is TRUE  → Included
Jamie  → Missing product 103 → NOT EXISTS is FALSE → Excluded
Taylor → No missing products → NOT EXISTS is TRUE  → Included
```

The outer `NOT EXISTS` checks **whether the middle query returns any rows**; it does not directly combine the inner Boolean values.

Another way to answer this query is using GROUP BY and HAVING.

```
SELECT c.customer_id, c.first_name, c.last_name
FROM Customer c
JOIN Purchase p
    ON c.customer_id = p.customer_id
JOIN RequiredProduct r
    ON p.product_id = r.product_id
GROUP BY c.customer_id, c.first_name, c.last_name
HAVING COUNT(r.product_id) = (
    SELECT COUNT(*)
    FROM RequiredProduct
);
```

Or another way to write it, using implicit JOIN is:

```
SELECT c.customer_id, c.first_name, c.last_name
FROM Customer c, Purchase p, RequiredProduct r
WHERE c.customer_id = p.customer_id
  AND p.product_id = r.product_id
GROUP BY c.customer_id, c.first_name, c.last_name
HAVING COUNT(r.product_id) = (
    SELECT COUNT(*)
    FROM RequiredProduct
);
```